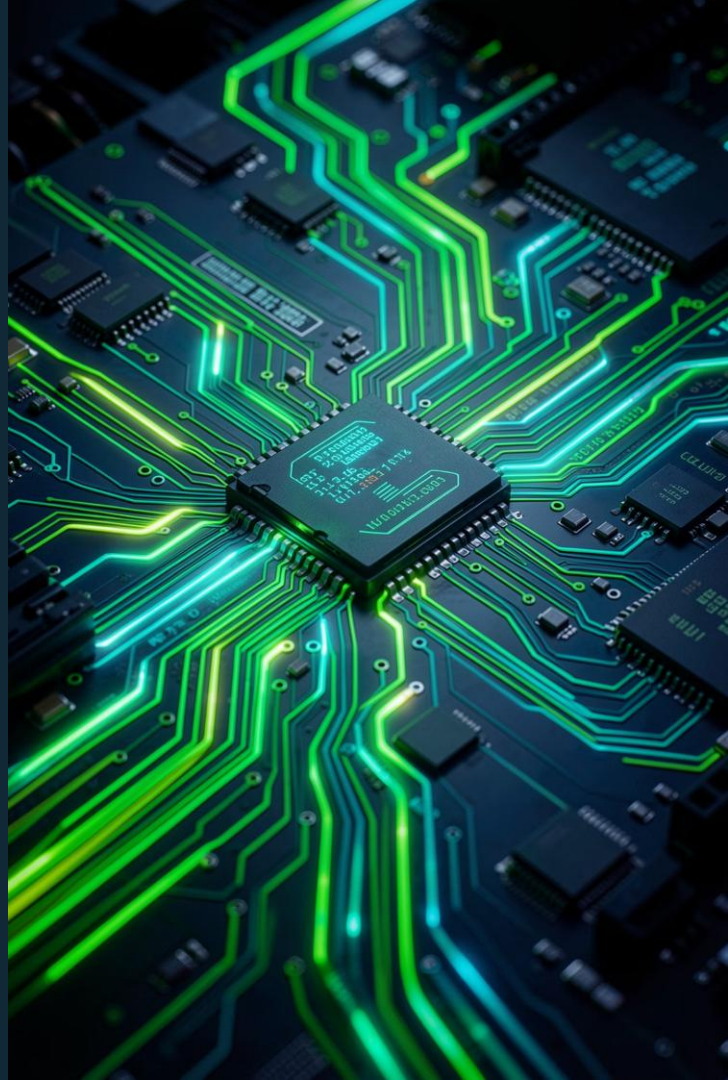
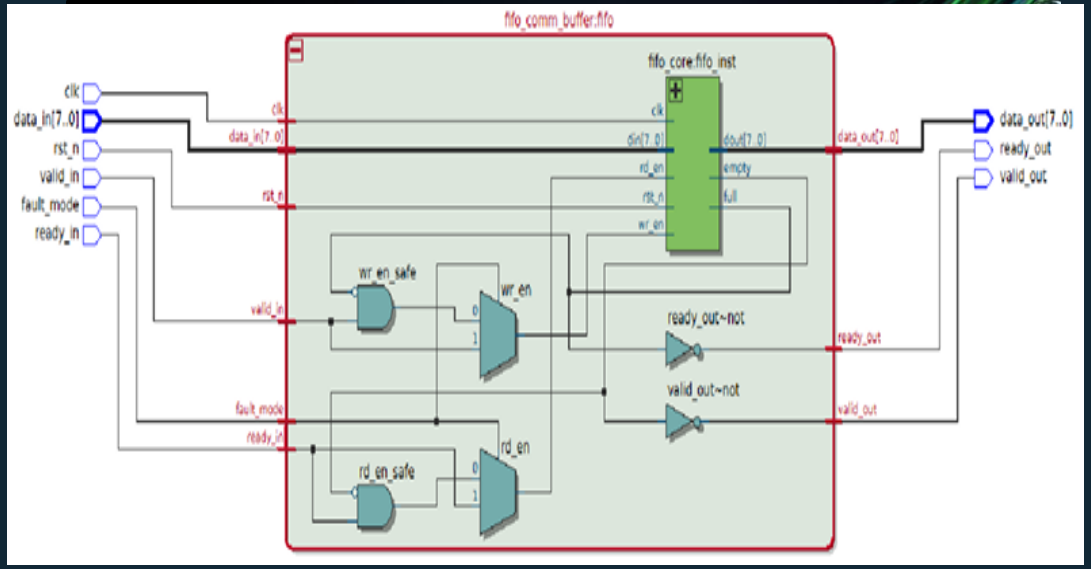
FPGA Implementation of a FIFO-Based Communication Buffer with Fault Injection Analysis

Avinash Tanti (252CN034) · M.Tech ECE(CEN)

Guide: Prof. Prashantha Kumar



System Overview



FIFO-Based Communication Buffer

Enables reliable data transfer under **rate mismatch** between producer and consumer.

FIFO ensures data ordering; **handshake ensures correctness**.

1

Producer

Data source

2

FIFO Buffer

Storage + control

3

Consumer

Data sink

RTL Architecture & Control Logic

Valid–Ready Control

Write: `valid_in & ~full`

Read: `ready_in & ~empty`

fifo_core

Memory + pointers

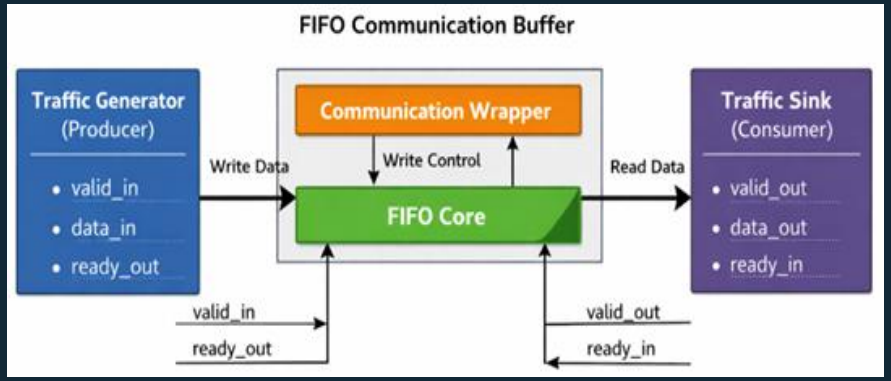
(wr_ptr, rd_ptr)

fifo_comm_buffer

Handshake + control logic

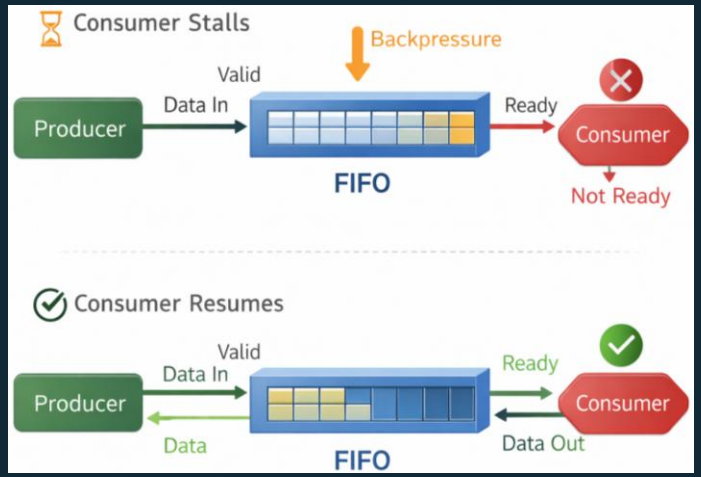
fifo_system_top

Integration layer



Key enhancement: **fault_mode** switches between safe (correct) and unsafe (intentional violation) control — separating data path from control path.

Progress Since Last Evaluation



Previously

FIFO + handshake design and simulation-level verification

Now Added

Fault injection · Behaviour comparison · Performance observation · FPGA hardware validation

Focus shifted from *design* → **validation & analysis**

Fault Injection Mechanism

How the system is intentionally broken via the `fault_mode` control signal.

⚠ **Effect:** Allows write when full and read when empty — used to study failure behaviour of the communication system.

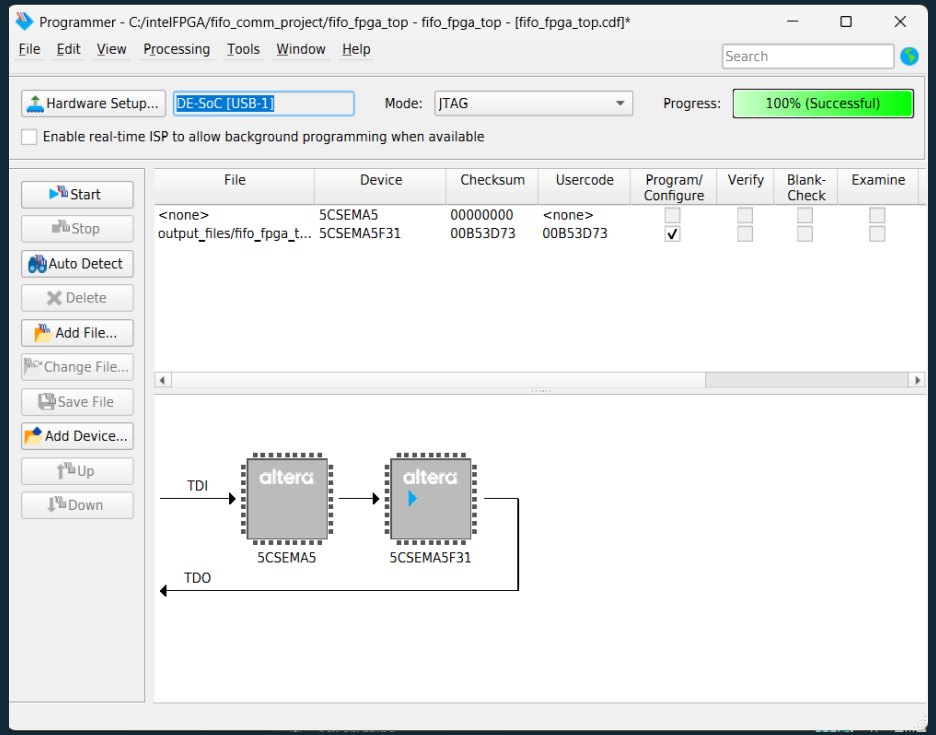
Normal Mode

```
wr_en = valid_in & ~fullrd_en = ready_in & ~empty
```

Fault Mode

```
wr_en = valid_inrd_en = ready_in
```

Simulation Setup & Observation

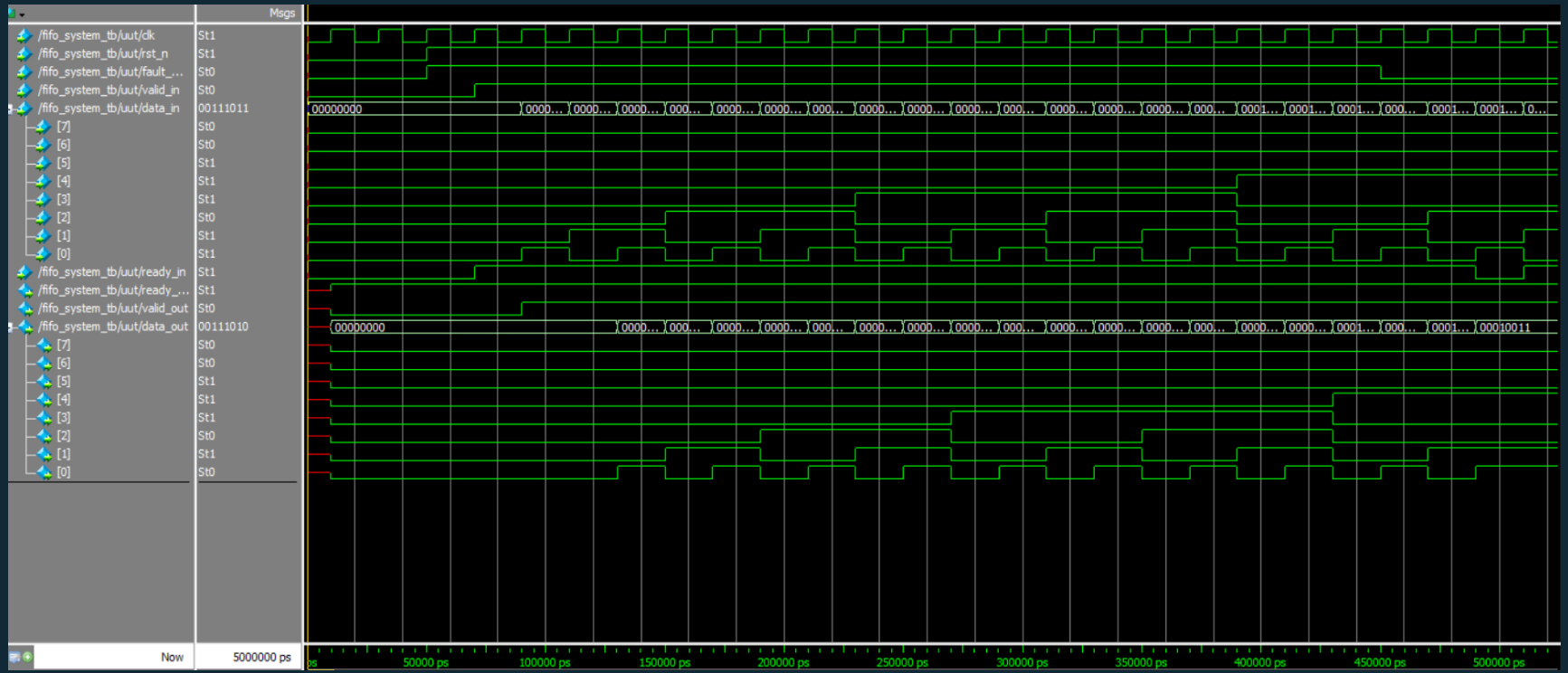


Verification Approach (ModelSim)

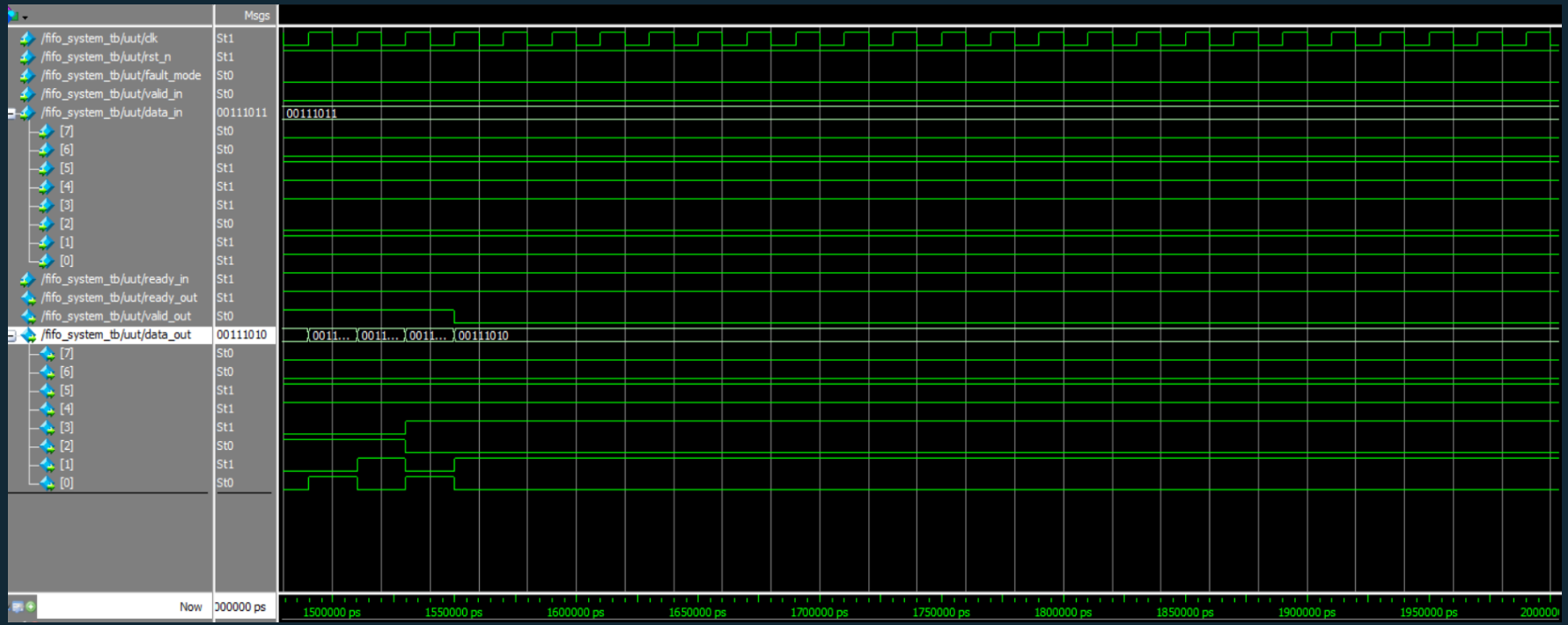
Single simulation run with **two sequential phases**: fault mode followed by normal mode.

- Continuous input data stream
- Variable readiness from consumer
- Realistic rate-mismatch conditions

Enables direct side-by-side comparison of correct vs incorrect behaviour.

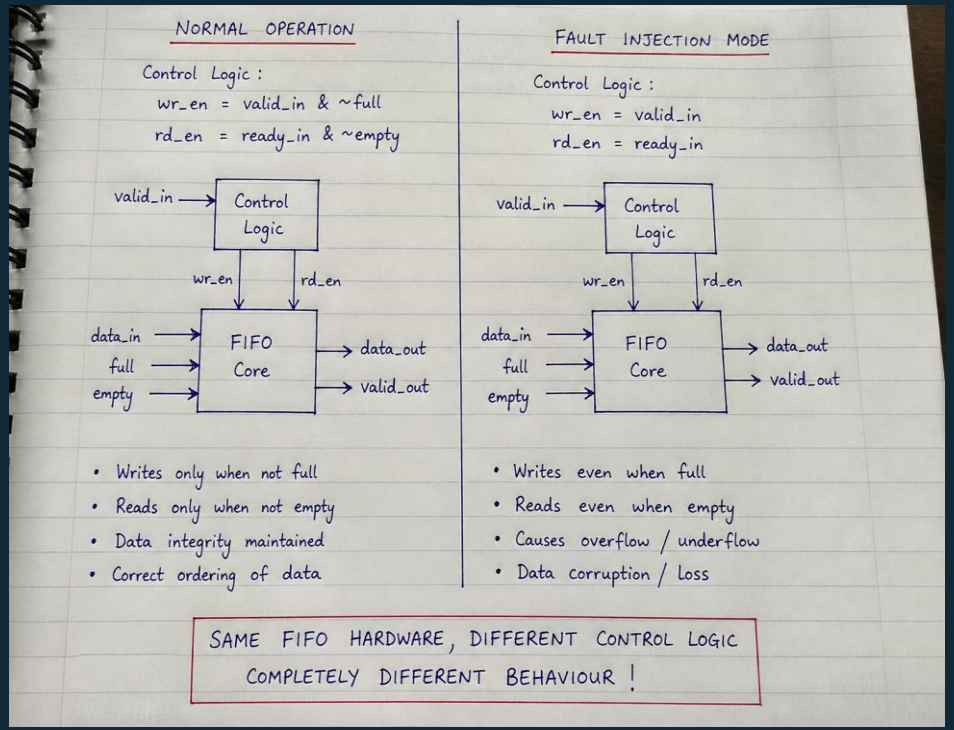


(a) Fault Mode Behavior



(b) Normal Mode Behavior

Why Fault Injection Matters



Same FIFO hardware. Different control logic. Completely different behaviour.

FIFO Guarantees

Data ordering and storage integrity

FIFO Does NOT Guarantee

Correct communication without proper control

Key Insight

System correctness depends on **control protocol**, not storage

Simulation Results & Performance Observation

⚠ Fault Mode

Data corruption · Invalid reads · Loss of ordering

✅ Normal Mode

Correct sequencing · Stable output

Only the control logic changed — the hardware is identical in both cases.

1–2
Clock Cycles
Latency per transaction in normal mode

1
Word/Cycle
Throughput in normal mode

⚠ In **fault mode**: throughput becomes irregular and the system loses reliability. Performance depends on **handshake availability**.

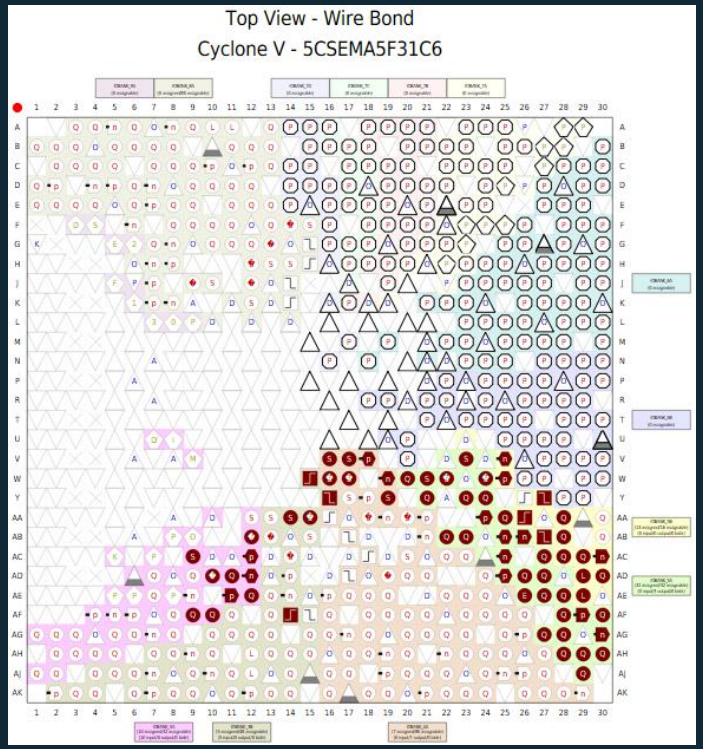
(a) Quartus Compilation Summary

Flow Summary

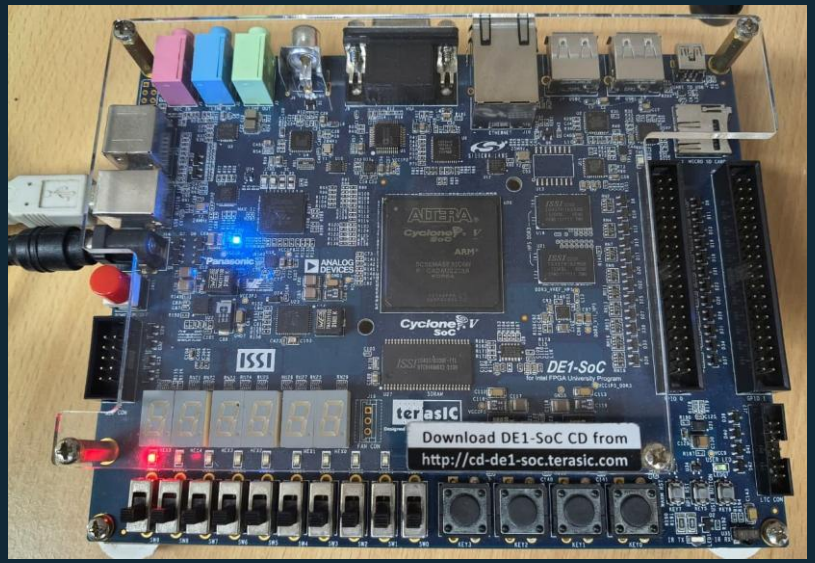
<<Filter>>

Flow Status	Successful - Sun Mar 29 20:13:38 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	fifo_system_top
Top-level Entity Name	fifo_system_top
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	21 / 32,070 (< 1 %)
Total registers	35
Total pins	23 / 457 (5 %)
Total virtual pins	0
Total block memory bits	128 / 4,065,280 (< 1 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

(b) FPGA Pin Mapping Configuration



FPGA Implementation & Hardware Validation

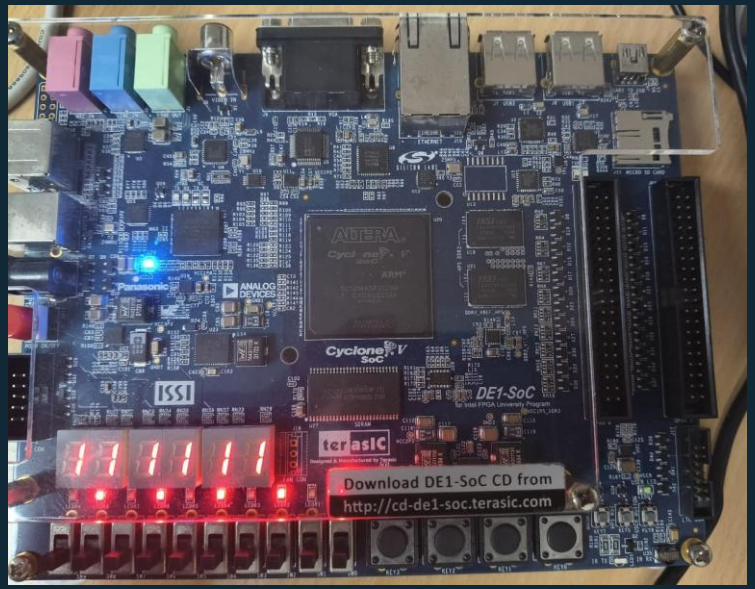


Cyclone V FPGA (DE1-SoC)

Synthesised and programmed successfully.

> Hardware behaviour matches simulation results exactly.

FPGA Implementation & Hardware Validation



FPGA Programming via JTAG Interface

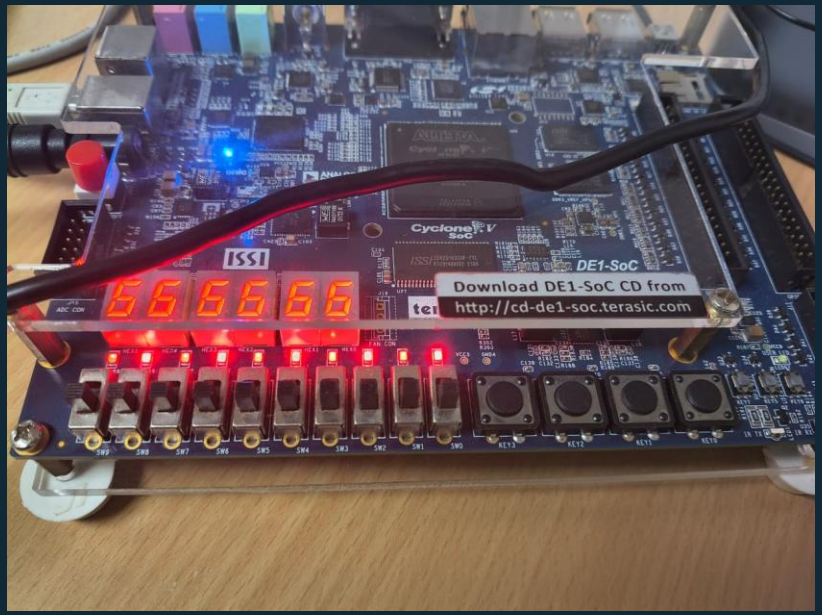
Inputs

Switches → input control signals

Outputs

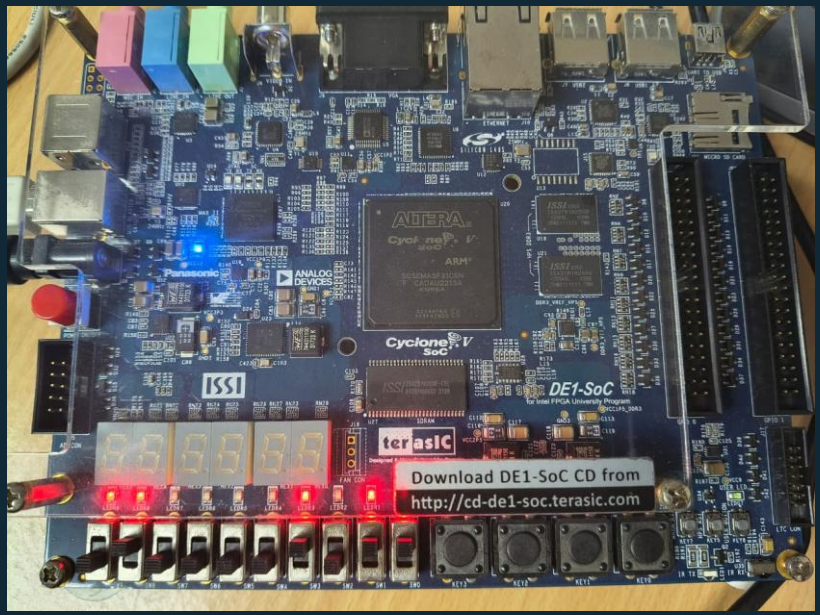
LEDs → observed output state

FPGA Hardware Output Observation



(a) Initial Output State

Switches → input control signals



(b) Output During Operation

LEDs → observed output state

Final Takeaways

Extended FIFO into a communication-aware system

Demonstrated failure under protocol violation

Verified behavior in simulation + hardware

Core learning:

Reliable digital systems depend on **correct control logic**, not just data storage.